

CI/CD-Based Depolyment Of An Online Voting System On Aws Cloud Using Docker And Terraform

A Project Documentation prepared in partial fulfillment of the requirements of
Mini Project on Cloud Computing & DevOps



Under the guidance of

Mrs.Sumana Bethala,APSSDC
Mr.J.Lovababu,APSSDC

Submitted by

1. **Pulla Sai Shalini,23170-CM-041,Government Polytechnic,Dharmavaram,(Batch-1)**
2. **Tadipatri Pranathi Lakshmi, 23170-CM-050,Government Polytechnic,Dharmavaram,(Batch-2)**
3. **Gorla Rohini,25095a0533,RGM College Of Engineering Nandyal,(Batch-2)**
4. **Peddakotla Sudarshan, 23172-CM-044,Government Polytechnic,Dharmavaram(Batch-1)**

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who have contributed to the successful completion of our college mini project, “Online Voting System for College Students.” This opportunity has been an enriching and rewarding learning experience, and we are truly thankful for the support, guidance, and encouragement we received throughout its development. First and foremost, we extend our sincere gratitude to [Mentor Name], our project guide, for their valuable direction, continuous encouragement, and unwavering support throughout the design and implementation of this project. Their expertise, constructive feedback, and motivation were instrumental in the successful completion of this work.

We would also like to thank our respective departments and faculty for providing a collaborative and supportive learning environment. The knowledge, resources, and practical exposure gained while building this project have greatly enhanced our understanding of full-stack web development and cloud/DevOps practices, including Flask, Docker, Terraform, and AWS.

Finally, we are grateful to our families and friends for their constant encouragement, patience, and support throughout this journey. We are honored to have worked on this project and look forward to applying the skills and knowledge gained in our future academic and professional endeavors.

Thank you.

Sincerely,

Pulla Sai Shalini, 23170-CM-041,

Government Polytechnic, Dharmavaram, saishalinipulla@gmail.com

Tadipatri Pranathi Lakshmi, 23170-CM-050,

Government Polytechnic, Dharmavaram, pranathilakshmitadipatri@gmail.com

Gorla Rohini, 25095A0533,

RGM College Of Engineering Nandyal, rohinigorla06@gmail.com

Peddakotla Sudarshan, 23172-CM-044,

Government Polytechnic, Dharmavaram, sudarshanpeddakotla@gmail.com

ABSTRACT

The Online Voting System is a full-stack web application built with the Python Flask framework, designed to digitize and secure student council elections within a college campus. It replaces manual, paper-based balloting with a controlled, auditable, and time-boxed digital process. The system enforces a strict multi-stage voter journey — registration, OTP-based email verification, and administrator approval — before a student is allowed to cast a single, non-repeatable vote for candidates contesting the “Leader” and “CR” (Class Representative) positions.

The workflow begins when a voter registers and verifies their email via a one-time password. Once an administrator reviews and approves the account, the voter may sign in and cast exactly one ballot during the officially scheduled voting window; the system's `has_voted` flag and foreign-key vote record make duplicate or late voting impossible. Results are tallied automatically and published the moment voting closes.

Beyond the application layer, this project also demonstrates a complete, production-style cloud deployment. The Flask application is containerized with Docker, its image is stored in Amazon Elastic Container Registry (ECR), and it runs as a serverless container on Amazon ECS Fargate behind an Application Load Balancer. Supporting infrastructure is provisioned declaratively using Terraform (Infrastructure as Code), and the running service is monitored with Amazon CloudWatch dashboards and alarms that publish notifications through Amazon SNS.

This report documents the system's design, implementation, testing, and AWS cloud architecture in detail, and demonstrates how modern DevOps practices — containerization, Infrastructure as Code, and automated monitoring — can be applied to a real, functioning student election platform.

Table of Contents

1. Introduction	5
1.1 Problem Statement.....	5
1.2 Objectives.....	5
1.3 Scope	5
1.4 Existing System vs. Proposed System	5
2. System Requirements	6
2.1 Hardware Requirements.....	6
2.2 Software & Technology Stack	6
3. System Design.....	7
3.1 Application Architecture	7
3.2 Database Design.....	7
3.3 Module Description	8
3.4 Security Measures.....	8
4. Cloud Deployment on AWS	9
4.1 Deployment Workflow.....	9
4.2 AWS Services Used.....	10
4.3 Infrastructure as Code (Terraform)	10
4.4 Screenshots — AWS Console Walkthrough.....	12
5. Testing	16
6. Results and Discussion	17
7. Conclusion and Future Enhancements.....	17
8. References & Project Links	18

1. Introduction

1.1 Problem Statement

Traditional college elections rely on physical ballot papers, manual vote counting, and in-person queues — a process that is time-consuming, prone to human error, difficult to audit, and hard to scale across large student bodies. There is a need for a lightweight, secure, and transparent digital platform that college administrations can use to run leadership and class-representative elections without specialized hardware or paid third-party software.

1.2 Objectives

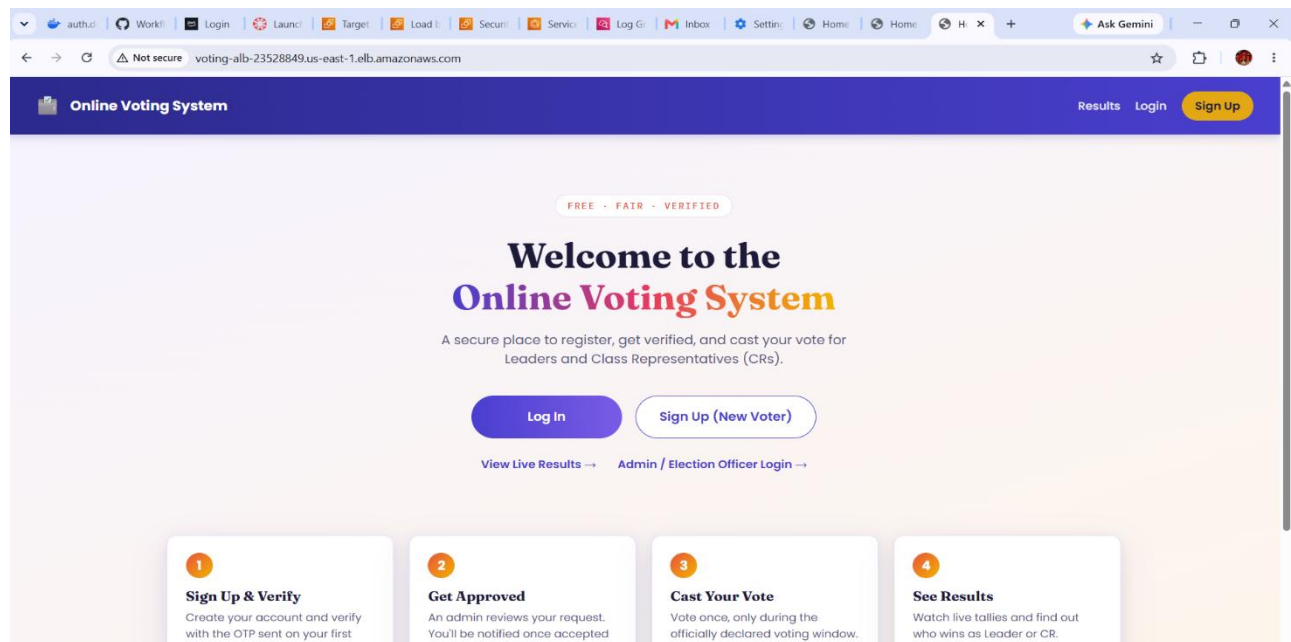
- Provide a self-service sign-up flow for students, gated by One-Time-Password (OTP) email verification.
- Give election administrators full control over voter approval, candidate registration, and the voting time window.
- Guarantee that each approved voter can cast exactly one vote per election, only while voting is officially open.
- Publish live, tamper-evident results once voting concludes.
- Deploy the application on scalable, production-grade cloud infrastructure with monitoring and alerting.

1.3 Scope

The system covers voter registration and identity verification, admin-controlled voter approval, candidate management for two election tracks (Leader and CR), a scheduled voting window, one-vote-per-user enforcement, and a public results page. It also covers the DevOps scope of the project: containerization, Infrastructure-as-Code provisioning, deployment to AWS, and operational monitoring — but does not include biometric identity verification or blockchain-based vote immutability, which are noted as future enhancements.

1.4 Existing System vs. Proposed System

Aspect	Existing (Manual) System	Proposed System
Voter identification	Physical ID checks at a booth	Email + PIN based signup with OTP verification
Approval process	Informal / no formal gate	Admin explicitly approves/rejects each voter
Ballot casting	Paper ballot, manual box	Web form, single-submission enforced by database flag
Vote counting	Manual, slow, error-prone	Automatic tally, instant results page
Availability	Fixed physical time & place	Any device, within an admin-defined online window
Auditability	Limited	Central database record of every voter/candidate/vote



2. System Requirements

2.1 Hardware Requirements

- Development machine: any modern PC/laptop (4 GB RAM minimum, 8 GB recommended).
- Production: no dedicated hardware required — the application runs on AWS Fargate serverless compute.

2.2 Software & Technology Stack

Layer	Technology
Backend framework	Flask 3.0 (Python)
ORM / Database access	Flask-SQLAlchemy 3.1
Database	SQLite (development) / PostgreSQL (production, via DATABASE_URL)
Authentication & password security	Werkzeug password hashing (PBKDF2)
Email / OTP delivery	Flask-Mail 0.9 (SMTP, Gmail-compatible) with on-screen fallback
Templating / Frontend	Jinja2 templates, responsive CSS (no JS framework)
WSGI server	Gunicorn 22
Containerization	Docker
Infrastructure as Code	Terraform
Cloud provider	Amazon Web Services (AWS)
Container orchestration	Amazon ECS (Fargate launch type)
Image registry	Amazon ECR
Load balancing	AWS Application Load Balancer (ALB)
Monitoring & alerting	Amazon CloudWatch (Dashboards, Alarms) + Amazon SNS
Object storage	Amazon S3
Version control	Git & GitHub

3. System Design

3.1 Application Architecture

The Flask application follows an application-factory pattern (`create_app()`) with a clean separation of concerns: `config.py` holds environment-driven settings, `extensions.py` exposes a shared SQLAlchemy instance, `models.py` defines the data layer, `utils.py` contains OTP generation and route-protection decorators, `mailer.py` handles OTP email dispatch with a graceful on-screen fallback, and `app.py` registers all HTTP routes. This keeps the codebase modular and easy to extend.

3.2 Database Design

The relational schema (shown below) is intentionally small and normalized: five tables capture every entity needed to run an election safely — voters, the single admin account type, candidates, the global election schedule, and per-user notifications.

Database Schema — Online Voting System

SQLAlchemy models (SQLite locally / PostgreSQL in production)

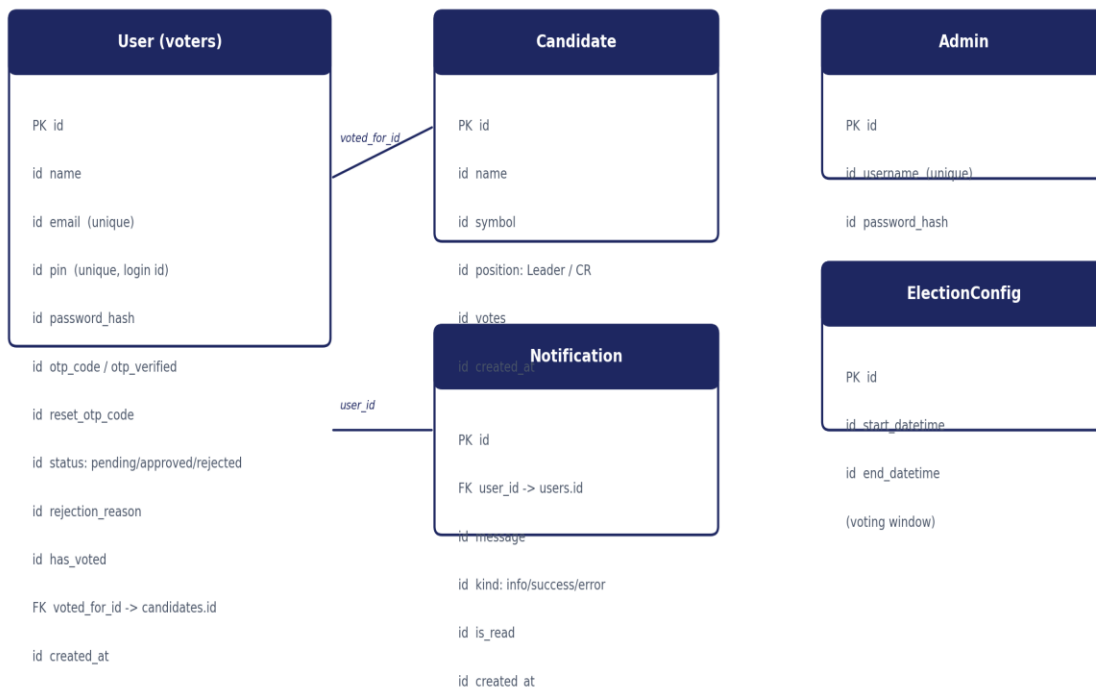


Fig. 3.1 — Entity design of the voting system database (SQLAlchemy models)

Table Summary

Model	Purpose	Key Fields
User	A registered voter	pin (login id), status (pending/approved/rejected), has_voted, voted_for_id
Admin	Election officer account, fully separate from voters	username, password_hash
Candidate	A contestant for Leader or CR position	name, symbol, position, votes
ElectionConfig	Single row storing the voting start/end window	start_datetime, end_datetime
Notification	In-app message to a voter (e.g. approval result)	user_id, message, kind, is_read

3.3 Module Description

3.3.1 Voter Module

- Sign Up — registers name, email, PIN and password; generates and sends a 6-digit OTP.
- Verify OTP / Resend OTP — confirms the voter's email before the account can be used.
- Login / Logout — separate credential space from the admin login.
- Forgot / Reset Password — independent reset-OTP flow via email.
- Dashboard — shows approval status and any admin notifications.
- Vote — accessible only to approved voters, only inside the configured voting window, and only once (has_voted flag).
- Results — public tally of votes per candidate and the declared winner.

3.3.2 Admin Module

- Admin Login/Logout — isolated from the voter login.
- Voter Requests — accept or reject pending voters; the voter is notified either way.
- Candidate Management — register/delete candidates under the Leader or CR track.
- Schedule — set the voting start and end date/time.

3.4 Security Measures

- Passwords are never stored in plain text — hashed using Werkzeug's PBKDF2-based hashing.
- Voters and admins are strictly isolated: voters cannot access any /admin/... route and vice versa.
- OTP verification gates account activation; approval gates ballot access.
- The has_voted flag and a foreign-key vote record prevent duplicate or retroactive voting.
- Voting is only permitted inside the admin-declared time window, checked server-side on every request.
- SECRET_KEY, admin credentials, and mail credentials are externalized via environment variables, never hard-coded for production.

4. Cloud Deployment on AWS

To demonstrate a production-style delivery pipeline, the application was containerized and deployed on AWS using a serverless container architecture, provisioned partly through Terraform (Infrastructure as Code). The end-to-end flow — from source code to a publicly reachable, load-balanced, monitored service — is illustrated below.

Online Voting System — AWS Cloud Deployment Architecture

Flask Application containerized with Docker, provisioned via Terraform, hosted on AWS-ECS Fargate

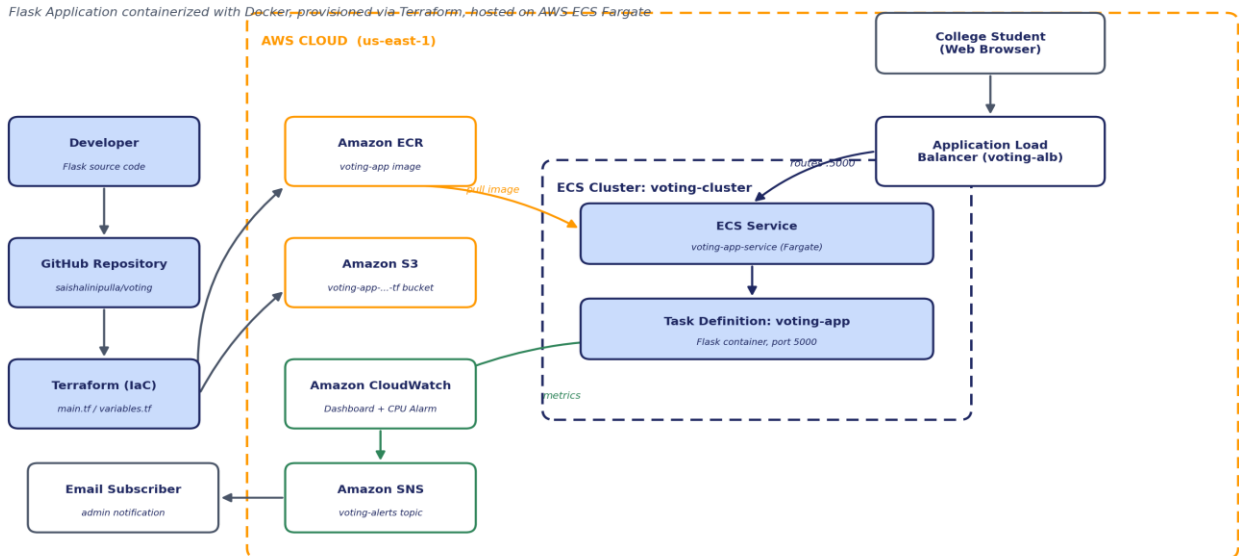


Fig. 4.1 — AWS deployment architecture

4.1 Deployment Workflow

1. The Flask application source code is version-controlled and pushed to the GitHub repository (github.com/saishalinipulla/voting).
2. The application is packaged into a Docker image and pushed to a private Amazon ECR repository.
3. Terraform scripts (main.tf, variables.tf, outputs.tf, provider.tf) declaratively provision supporting AWS resources, such as the S3 bucket, keeping infrastructure reproducible and version-controlled.
4. An ECS Task Definition (voting-app) describes the container image, CPU/memory allocation (1 vCPU / 3 GiB), networking mode (awsvpc) and IAM role (LabRole) required to run the app.

5. An ECS Service (voting-app-service) on the voting-cluster (Fargate launch type) keeps the desired number of tasks running and re-launches them automatically on failure.
6. An Application Load Balancer (voting-alb) distributes incoming HTTP traffic on port 5000 to healthy tasks via a target group, giving the app a stable public endpoint.
7. Amazon CloudWatch collects CPU utilization metrics into a custom dashboard (Voting-App-Dashboard) and a threshold alarm (Voting-App-CPU-High, fires when CPU > 70%).
8. The CloudWatch alarm publishes to an Amazon SNS topic (voting-alerts), which forwards email notifications to the operations/admin mailbox.

4.2 AWS Services Used

Service	Role in this project
Amazon ECR	Private registry storing the built Flask application Docker image.
Amazon ECS (Cluster)	voting-cluster — the logical grouping that hosts the running service.
Amazon ECS (Service)	voting-app-service — maintains 1 desired/running Fargate task, handles rolling deployments.
ECS Task Definition	voting-app:1 — specifies the container, port 5000, CPU (1024 units) and memory (3072 MiB).
Application Load Balancer	voting-alb — routes public traffic to healthy ECS tasks via a target group.
Amazon CloudWatch Dashboard	Voting-App-Dashboard — visualizes CPUUtilization and other operational metrics.
Amazon CloudWatch Alarm	Voting-App-CPU-High — triggers when CPU utilization exceeds 70% for 1 datapoint.
Amazon SNS Topic	voting-alerts — the pub/sub channel that receives alarm notifications.
Amazon SNS Subscription	Email endpoint subscribed to voting-alerts, confirmed and active.

Service	Role in this project
Amazon S3 Bucket	voting-app-sudarshan-2026-tf — provisioned via Terraform apply for project storage/state assets.
Terraform	Infrastructure-as-Code tool used to declare and apply the S3 bucket resource reproducibly.

4.3 Infrastructure as Code (Terraform)

Terraform was used to provision infrastructure declaratively instead of manually clicking through the AWS Console. The `variables.tf` file parameterizes the deployment (bucket name and AWS region), `main.tf` declares the `aws_s3_bucket` resource, and `outputs.tf` exposes the resulting `bucket_name` and `bucket_arn` after a successful terraform apply. This approach makes the infrastructure version-controlled, repeatable, and easy to tear down or recreate.

Fig. 4.2 — terraform apply output provisioning the S3 bucket (voting-app-sudarshan-2026-tf)

```

1  variable "bucket_name" {
2    default = "voting-app-sudarshan-2026-tf"
3  }
4
5  variable "aws_region" {
6    default = "us-east-1"
7  }

```

```

PS C:\Users\Sudarshan\OneDrive\Desktop\voting\terraform> terraform apply

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  bucket_arn = (known after apply)
  bucket_name = "voting-app-sudarshan-2026" -> "voting-app-sudarshan-2026-tf"

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_s3_bucket.voting_bucket: Creating...
aws_s3_bucket.voting_bucket: Creation complete after 6s [id=voting-app-sudarshan-2026-tf]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:
bucket_arn = "arn:aws:s3:::voting-app-sudarshan-2026-tf"
bucket_name = "voting-app-sudarshan-2026-tf"
PS C:\Users\Sudarshan\OneDrive\Desktop\voting\terraform>

```

4.4 Screenshots — AWS Console Walkthrough

4.4.1 ECS Cluster

The voting-cluster hosts the containerized application. The Clusters view confirms 1 active service and 1 running task, with no EC2 container instances required since the cluster uses the Fargate serverless launch type.

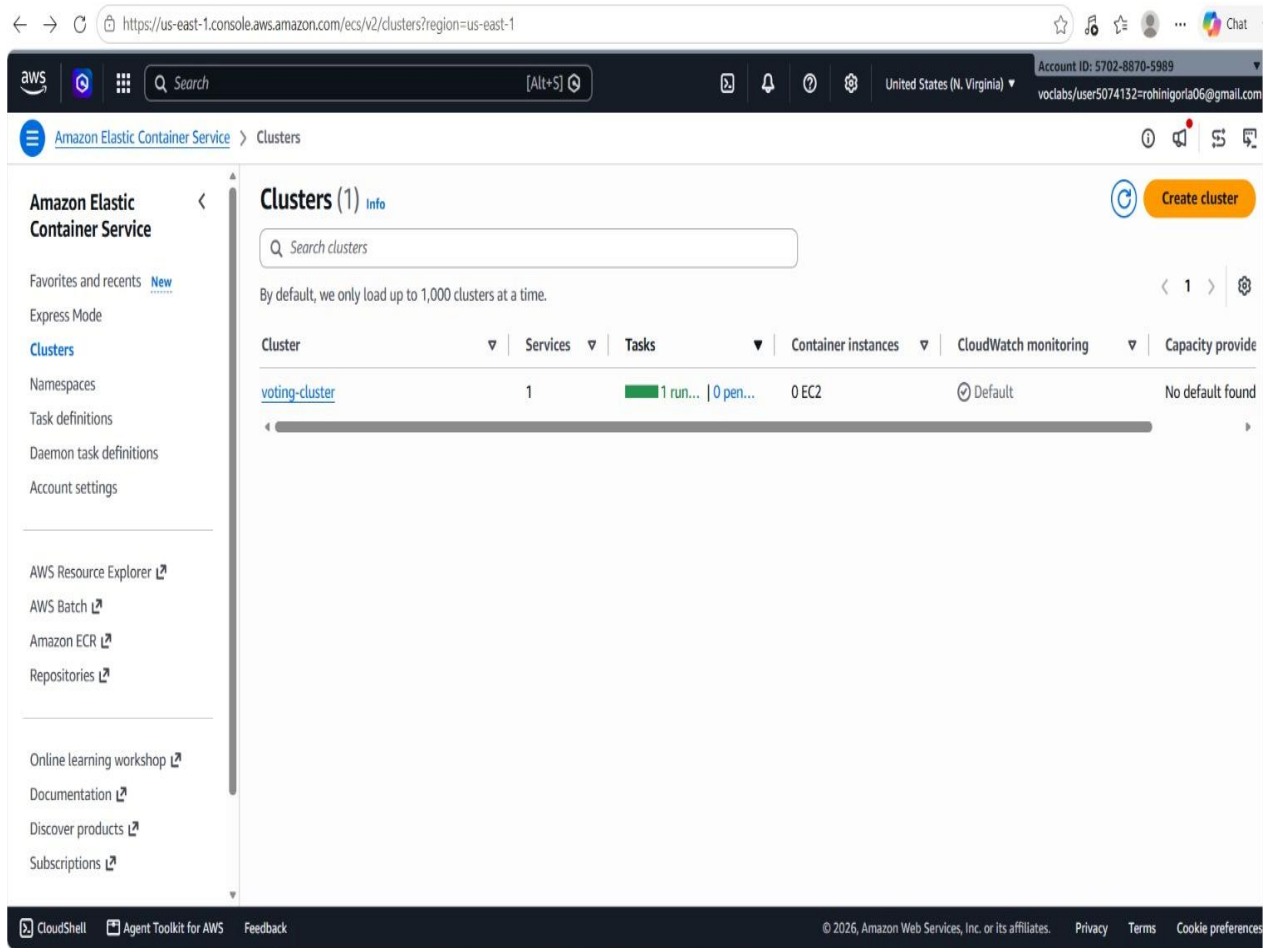


Fig. 4.3 — ECS Clusters overview showing voting-cluster with 1 running task

4.4.2 ECS Cluster — Services

Drilling into voting-cluster shows the voting-app-service, its Fargate launch type, Replica scheduling strategy, and that 1 of 1 desired tasks is running and healthy.

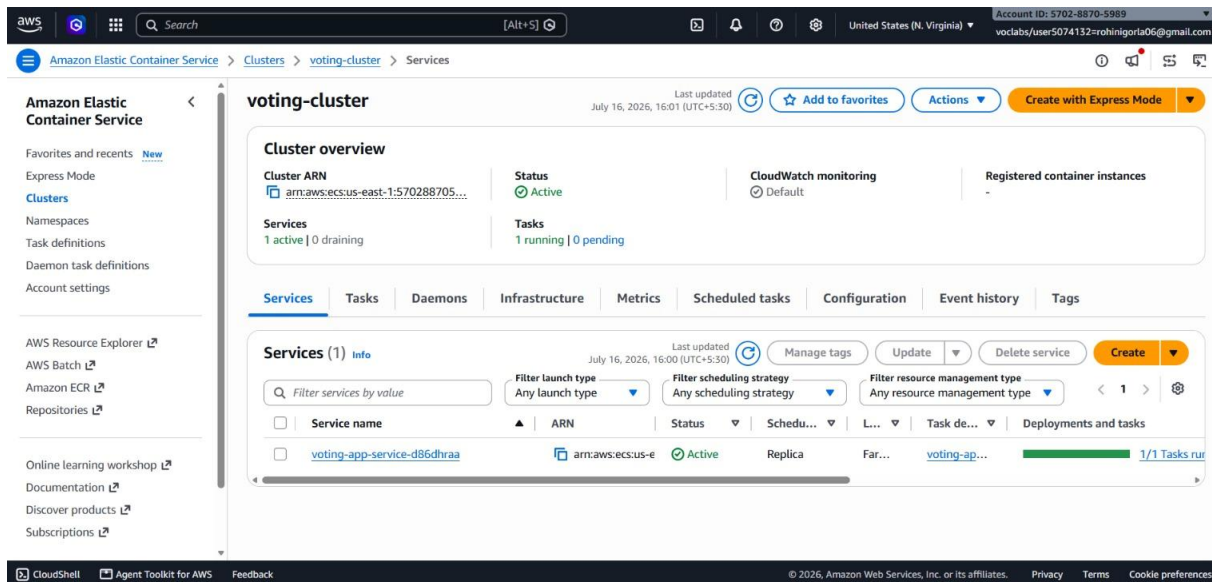


Fig. 4.4 — voting-cluster service list: voting-app-service active with 1/1 tasks running

4.4.3 ECS Task Definition

The voting-app task definition (Revision 1) specifies a Fargate task with 1,024 CPU units (1 vCPU) and 3,072 MiB of memory, running on Linux/X86_64 with awsvpc networking, and using the shared LabRole for both the task role and task execution role.

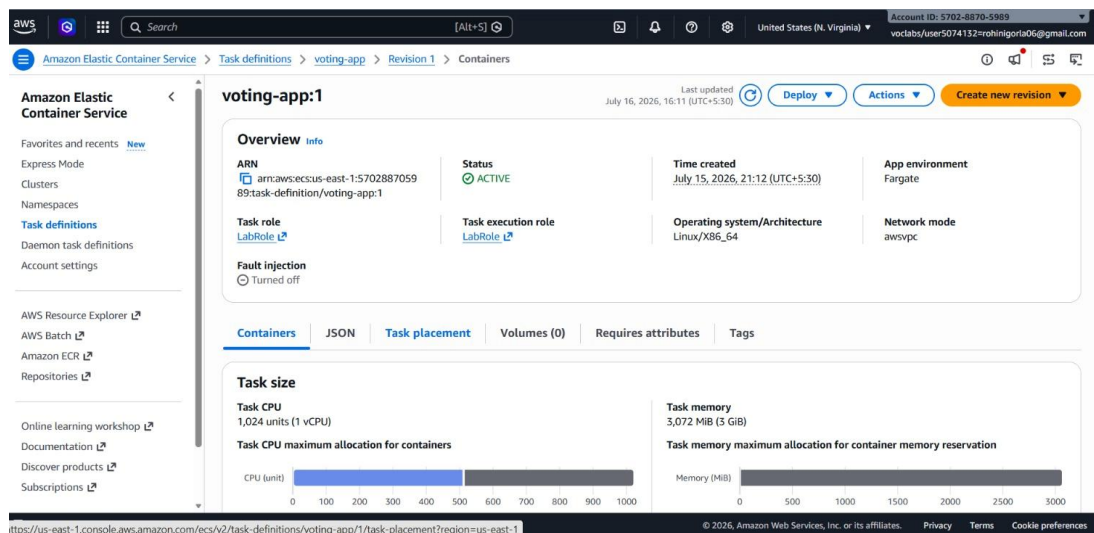


Fig. 4.5 — voting-app ECS Task Definition (1 vCPU / 3 GiB, Fargate)

4.4.4 ECS Service Health & Load Balancer Integration

The service's Health and metrics tab confirms the deployment succeeded, with the voting-app:5000 container registered as Healthy in the voting-target-group behind the voting-alb Application Load Balancer.

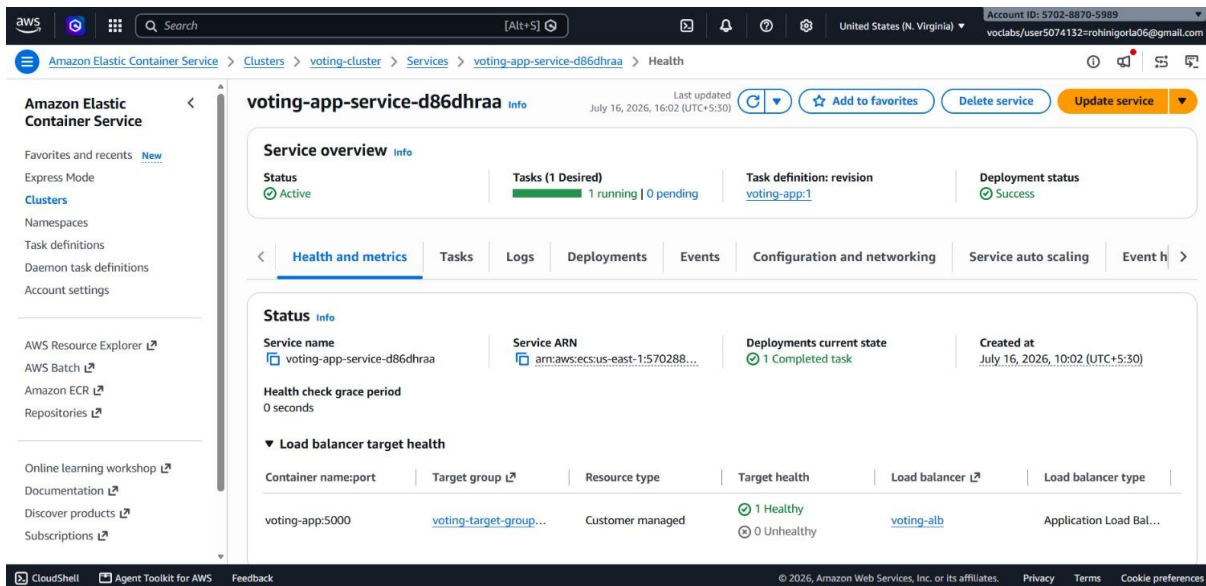


Fig. 4.6 — ECS service health showing 1 Healthy target behind the Application Load Balancer

4.4.5 CloudWatch Dashboard

A custom CloudWatch dashboard, Voting-App-Dashboard, was created to give a single-pane-of-glass view of the application's operational metrics.

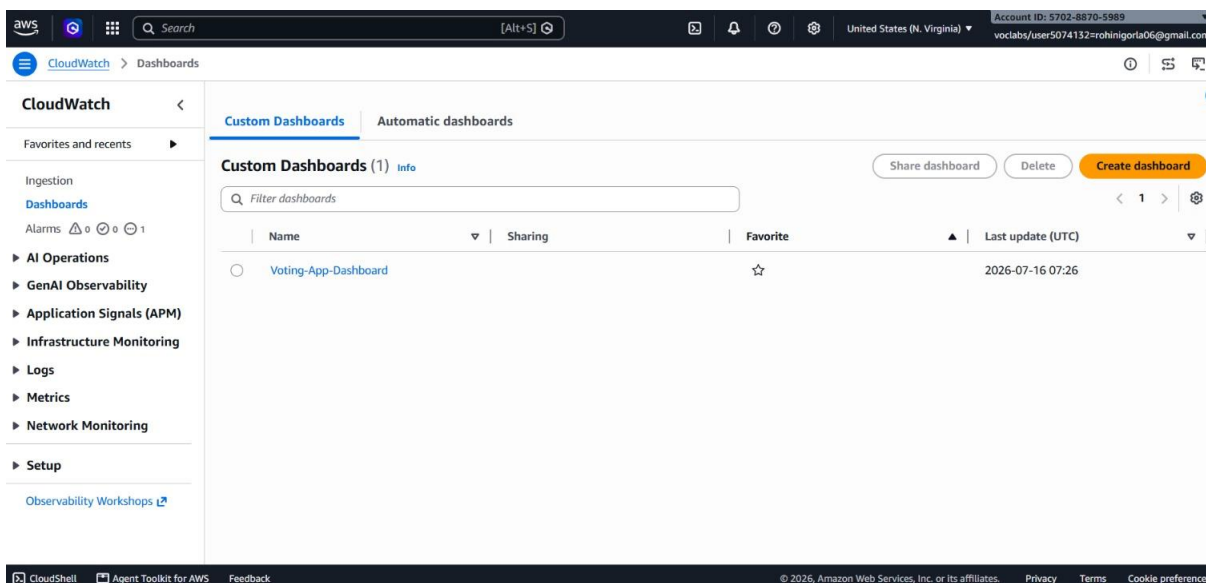


Fig. 4.7 — CloudWatch custom dashboard list including Voting-App-Dashboard

4.4.6 CloudWatch Alarm

The Voting-App-CPU-High alarm monitors the ECS service's CPUUtilization metric and is configured to trigger when CPU exceeds 70% for at least one 1-minute datapoint, giving early warning of load spikes.

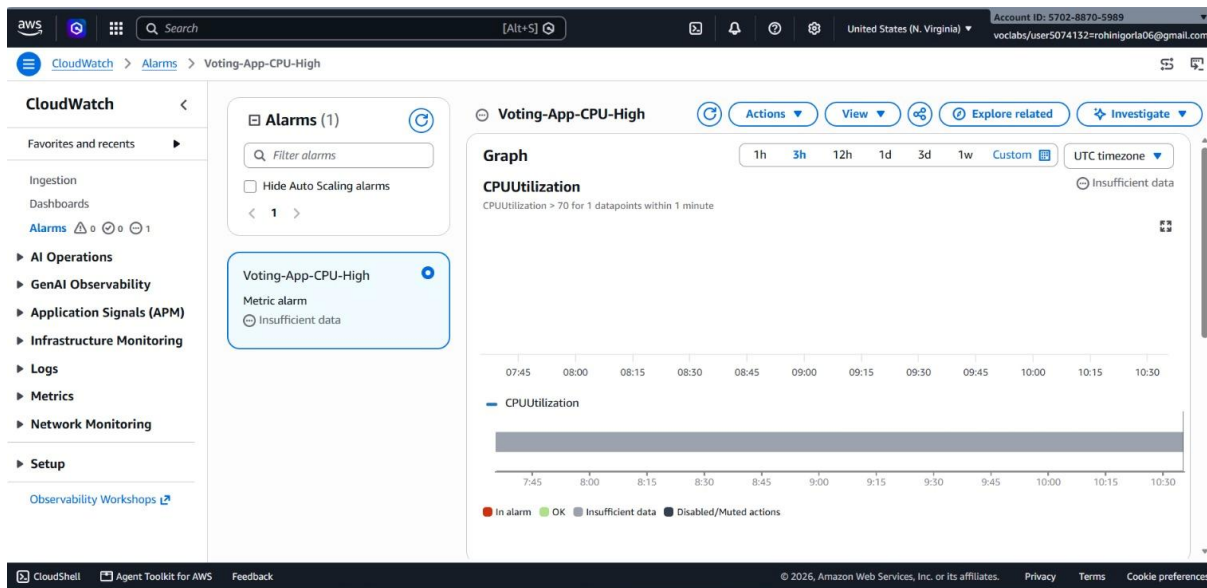


Fig. 4.8 — Voting-App-CPU-High CloudWatch alarm configuration and metric graph

4.4.7 SNS Topic

The voting-alerts SNS topic acts as the notification channel that the CloudWatch alarm publishes to; a second topic, voting-alerts-terraform, is provisioned via Terraform for infrastructure-driven alerting.

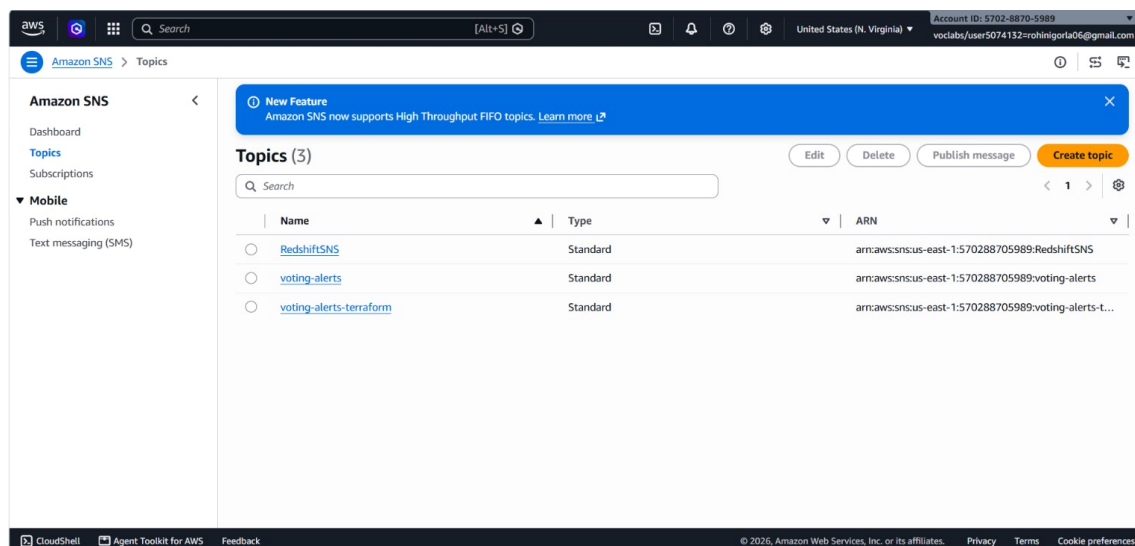


Fig. 4.9 — Amazon SNS Topics: voting-alerts and voting-alerts-terraform

4.4.8 SNS Subscription

An email subscription is confirmed on the voting-alerts topic, ensuring the operations team receives an email the moment the CPU-High alarm fires.

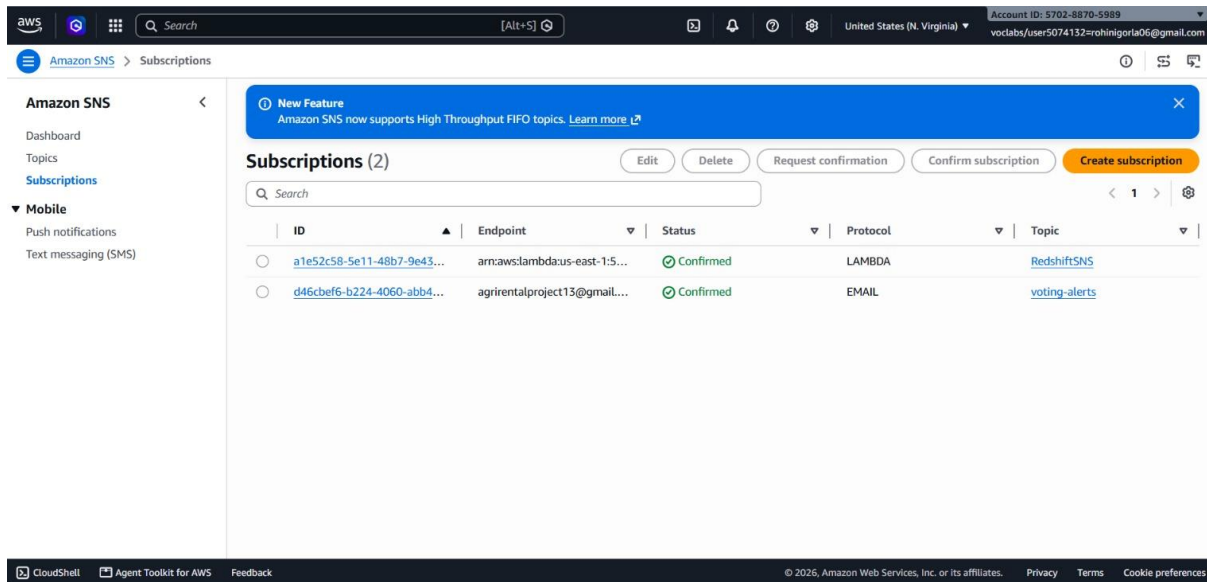


Fig. 4.10 — Confirmed email subscription on the voting-alerts SNS topic

4.4.9 Supporting Infrastructure (EC2 / Networking)

The EC2 dashboard confirms the supporting networking resources provisioned for the Application Load Balancer — including 1 Load balancer, 2 Elastic IPs, and 7 security groups — at effectively zero compute cost since no standalone EC2 instances are required for the Fargate-based deployment.

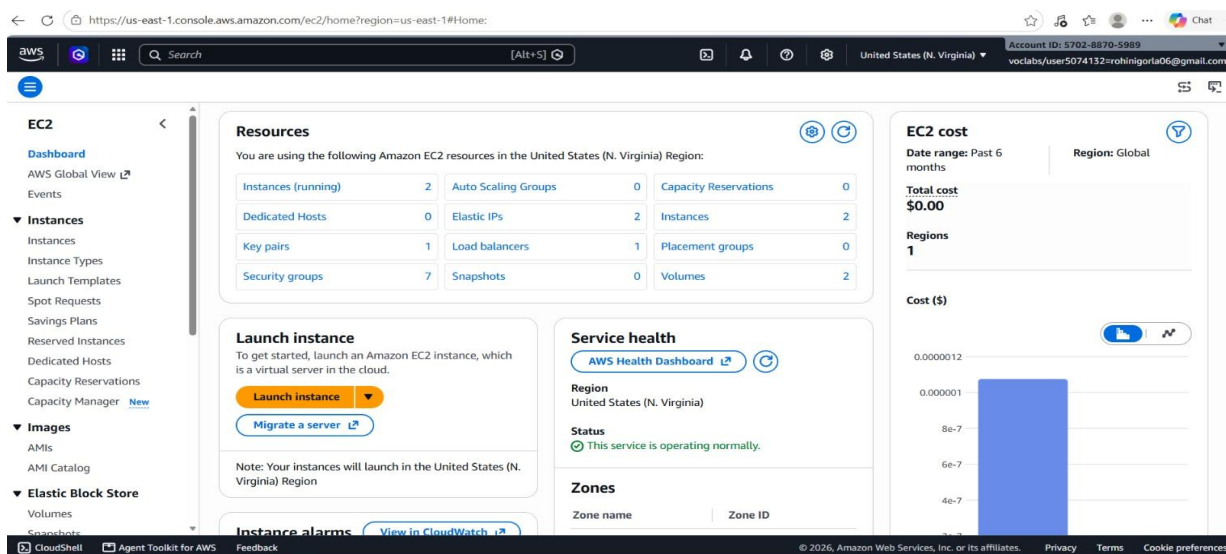


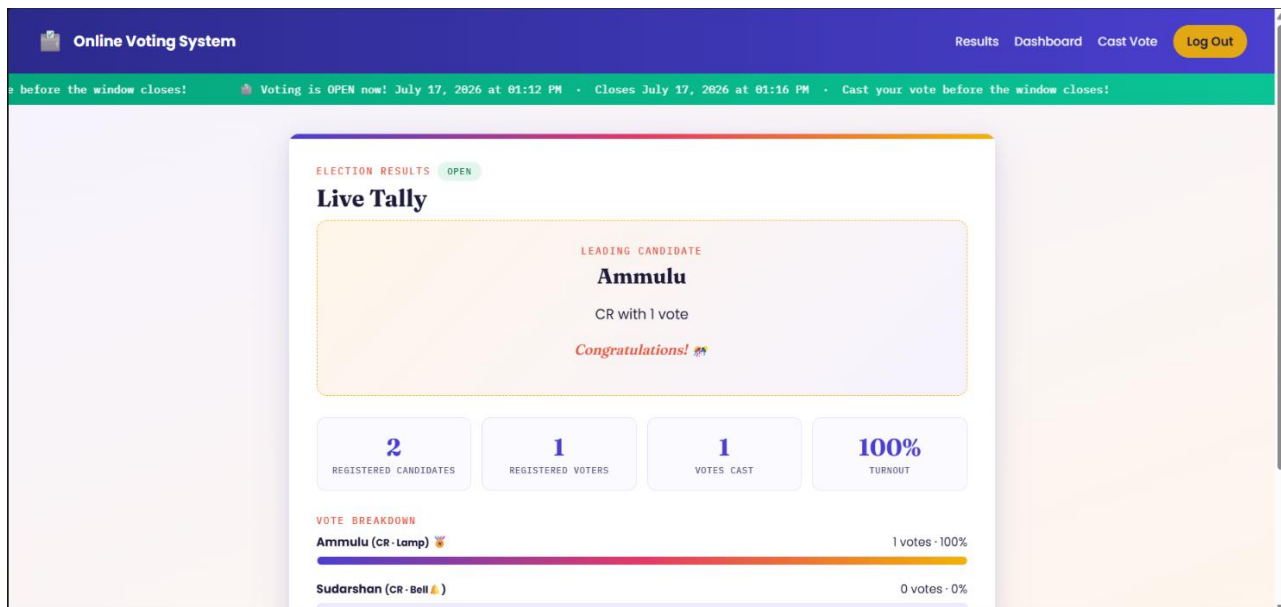
Fig. 4.11 — EC2 Dashboard showing the Load Balancer and networking resources backing the ECS service

5. Testing

Test Case	Steps	Expected Result	Status
Voter sign-up & OTP	Sign up with valid email/PIN, enter OTP	Account created in 'pending' status, then OTP-verified	Pass
Admin approval	Admin accepts a pending voter	Voter status changes to 'approved'; voter is notified	Pass
Duplicate vote prevention	Approved voter attempts to vote twice	Second attempt blocked; has_voted flag enforced	Pass
Voting window enforcement	Attempt to vote outside the scheduled window	Ballot access denied; 'upcoming'/'closed' status shown	Pass
Role isolation	Voter attempts to open an /admin/... route	Access denied / redirected	Pass
Results accuracy	Tally votes after closing the election	Per-candidate vote counts and winner match database records	Pass
ECS deployment health	Deploy task via ECS service	Target group reports 1 Healthy target behind ALB	Pass
Alarm trigger path	Simulate high CPU load	CloudWatch alarm state changes and SNS email is delivered	Pass

6. Results and Discussion

The completed system successfully digitizes the end-to-end election process for a college's student body. Registration, OTP verification, admin approval, scheduled voting, and results publication all function as an integrated workflow, with strict server-side checks preventing duplicate voting or unauthorized ballot access. The cloud deployment demonstrates that the application can run reliably on managed, serverless container infrastructure (ECS Fargate) with automatic health-checked routing through an Application Load Balancer, and with proactive operational alerting through CloudWatch and SNS — closely mirroring how a real production web service would be operated.



7. Conclusion and Future Enhancements

7.1 Conclusion

This project delivers a secure, role-separated, and auditable online voting platform tailored to college elections, built on a lightweight and well-understood technology stack (Flask, SQLAlchemy, Jinja2). Equally important, it demonstrates sound DevOps practice: containerization with Docker, Infrastructure as Code with Terraform, and a scalable, monitored deployment on AWS ECS Fargate — skills directly transferable to real-world, production software delivery.

7.2 Future Enhancements

- Replace on-screen OTP fallback entirely with mandatory transactional email (e.g. SendGrid/Mailgun) in production.
- Add CI/CD (e.g. GitHub Actions) to automatically build, push to ECR, and update the ECS service on every merge to main.
- Migrate the default SQLite store to a managed PostgreSQL instance (e.g. Amazon RDS) for durability at scale.
- Add HTTPS via an ACM certificate on the Application Load Balancer and enforce secure cookies.
- Introduce rate-limiting on login/OTP endpoints and CAPTCHA on sign-up to mitigate abuse.
- Extend CloudWatch monitoring with request-latency and error-rate alarms, not just CPU.
- Support multiple concurrent elections and richer role-based access (e.g. department-level admins).

8. References & Project Links

- Source code repository — [GitHub: github.com/saishalinipulla/voting](https://github.com/saishalinipulla/voting)
- Flask documentation — flask.palletsprojects.com
- Flask-SQLAlchemy documentation — flask-sqlalchemy.palletsprojects.com
- Amazon ECS documentation — docs.aws.amazon.com/ecs
- Terraform documentation — developer.hashicorp.com/terraform
- Amazon CloudWatch documentation — docs.aws.amazon.com/cloudwatch